

職務経歴書

最終更新: 2026/08/09

プロフィール

基本情報

項目	内容
名前	おくの
経験職種	小規模Webアプリ開発（FE, BE, PaaSインフラなど）
実務年数	1年8ヶ月
職務経歴サイト	 仮サイト (https://github.com/200okmk/resume/tree/main/public#readme)
16タイプ	INTP論理学者

資格

資格名	取得日
AWS認定 SAA	2026年8月（取得予定）
IPA 基本情報技術者試験	2026年5月
証券アナリスト1次、外務員一種など金融系	2021年

アカウント

- [Github](#)

職務要約

現場実務で約1年半、TypeScript言語での小規模なWebアプリ/システムの開発業務に従事。**基本単独でフロントエンドFE, バックエンドBE, インフラ層を横断的に担当**してきました（開発職の先輩が在籍中は適宜相談してご協力頂きました）。UX向上や将来性の視点も持ちながら、機能仕様やUI改善などを自ら検討・提案してきました。具体的な主業務は**詳細仕様の検討と提案、技術選択、各種設計、FEとBEの実装、ドキュメント作成、公開、継続保守など開発業務全般**です。その他並行業務としてWeb制作案件ごとにホスト先PaaSとDNS管理、脆弱性発表への対応、既存開発案件の引き継ぎ保守、自社ブログの執筆、エンジニア採用面接なども担当していました。今後はキャリアの軸足の中・大規模プロダクトのバックエンドBE開発にステップアップさせるべく、新たにJava言語とSpring、またAWSの資格勉強に励んでいます。

技術スキル

個人学習した技術も含めています。

フロントエンドFE

HTML , TailwindCSS , CSS , TypeScript , Astro , React19 , TanStack Start/Router , Next.js AppRouter , Shadcn/ui , RadixUI , Zustand , TanStack Query/Table/Form , ReactHookForm

バックエンドBE

Node.js(TS) , Hono , NestJS , BetterAuth , Clerk , DrizzleORM , PrismaORM , Java , SpringBoot , SQL , SQLite , PostgreSQL , NeonDB , Supabase ORM/DB/Auth/Storage , Slack API , Notion API

PaaS / AWS

Cloudflare Workers/Pages/D1/DNS , Vercel , Netlify , Render , AWS Amplify , IAM , Lambda , S3 , S3 Vectors , Bedrock KnowledgeBase , Bedrock Agents , EventBridge , SES

その他 / DevOps

Zod , ESLint , Prettier , Biome , Husky , Github Actions , Docker , DevContainer , Mermaid , Turborepo , GraphQL

ツール

MacOS , Zsh , Ghostty , Zed , Cursor , VSCode , Eclipse , Github , Notion , Slack , MicroCMS , Figma

職務経歴

会社ごと、プロジェクトごとに**時系列降順**で詳述

1. 千葉県のWeb制作会社【2024/07 - 2026/02】

Next.js AppRouter でのWebアプリ開発に即参加できる会社のみに絞った就活を経て、エンジニアキャリアを始めた最初の会社です。TypeScript言語での小規模なWebアプリ・Webシステムの開発業務に従事しました。慢性的な開発リソース不足もあり、**基本単独でFE, BE, インフラ層を横断的に担当**してきました（開発職の先輩が在籍中は適宜相談してご協力頂きました）。UX向上や将来性の視点も持ちながら、機能仕様やUI改善などを自ら検討・提案してきました。

項目	内容
事業内容	主に企業HP受託制作、Web/SNSマーケティング、記事コンテンツ制作、Webアプリ受託開発
取引業態	一次請け直接取引
社員数	11名前後（うちマークアップエンジニア2〜3名, 開発エンジニア1〜2名）
主な担当業務	受託および自社の開発案件において、詳細仕様の検討と提案、技術選択、各種設計、FEとBEの実装、ドキュメント作成、公開、継続保守など開発業務全般
その他並行業務	Web制作案件ごとにホスト先PaaSとDNS管理、脆弱性発表への対応、既存開発案件の引き継ぎ保守、自社ブログ執筆、エンジニア採用面接など

最も困難だったプロジェクトは、入社直後に参加したフルスタックWeb開発案件（1-1）です。

1-3. 自社Webサイト用RAGチャットボット開発【2025/07 - 2025/11】

概要

既存自社Webサイトのページ右下に配置する質疑応答RAGチャットボットの社内開発。高い学習・実装コストによる属人リスクや工期増加、他リポジトリへの再現性なども考慮し、RAG機構本体は自前実装せずにフルマネージドサービス Amazon Bedrock を採用しました。大まかなアーキテクチャは、チャットインターフェースIFからのユーザー入力をAstroバックエンド経由でBedrockのAPIに渡し、その返り値を1文字ずつストリーム表示するシンプルなものです。またRAG関連のAWS使用料金を毎日定刻にSlack通知してコストを継続的に監視できる仕組みも Lambda で構築しました。筆者が単独で要件定義から実装まで全て担当したのち、応答情報源マークダウンの作成を担当するチームに引き継いで終了。

使用技術

- フロントエンドFE： Astro , RadixUI(AssistantUI Primitive) , CSS ,
- バックエンドBE： Astro , Bedrock KnowledgeBase , Bedrock Agents , S3 , S3 Vectors , IAM , EventBridge , Lambda , AWS CostExplorer API , Slack API , Slack BlockKit
- インフラ/その他： Cloudflare Pages (Webサイト本体)

チーム体制

役割	人数	担当業務	備考
開発エンジニア（筆者）	1名	要求整理と要件定義、調査と技術選定、アーキテクチャ設計、FE・BE実装、ドキュメント作成など開発業務全般	筆者が単独で担当。他業務を優先し合間で進行

筆者の担当業務詳細

要件定義

- 口頭での要求をヒアリング・整理
- 強制したいRAG応答要件
 - 事前に与えた情報のみ参照
 - 日本語のみで返答など
- コスト通知機構の要件、表示内容など（下記①）
- チャットIFのUIと機能仕様（下記②）
- チャット専用全画面ページの作成

調査と技術選定、アーキテクチャ設計

- RAG技術、ベクトルDBの概要を学習
- 将来性を考慮した技術選定
 - RAG機構本体は自前実装ではなくフルマネージドサービスを利用（属人リスクや他リポジトリへの再現性から）
 - フルマネは Google Vertex ではなく Amazon Bedrock を採用（ネット上の知見数やAWS利用実績から）
 - FE実装にて「機能的なチャットIFの迅速な構築」と「各サイトデザインに完全適応できるスタイリング柔軟性」の両立
- コストを優先した Bedrock KnowledgeBase 構成選択
 - ストレージ（ S3 ）
 - ベクトルDB（ S3 Vectors ）
 - ベクトル変換モデル（ Cohere 多言語版 ）
 - 応答LLMモデル（ Claude 廉価モデル ）
 - その他
- Bedrock Agent を追加導入して応答要件の安定化
 - 上記KBをラップして自然言語での詳細なカスタムプロンプトを付与
 - チューニングの反復
- ①RAGコスト監視機構
 - EventBridge で毎日定刻に Lambda を起動して下記を実行
 - 各種APIから日次料金を取得、月次累計など計算
 - Slack BlockKit でUI構築、数値をはめ込み
 - Slack専用チャンネルに通知、S3に保存

②FE実装（よくある開閉ポップオーバー式チャットIF）

- BEからの応答を1文字ずつ随時ストリーム表示
- 会話の保持とリセット
- チャット内自動スクロールダウン
- 状態管理と表示
- 音声読み上げ・キーボード操作などのアクセシビリティ対応
- FAQ3つを事前にボタン化してチャット上部に配置
- モバイルブラウザ専用のUI挙動調整

ドキュメント作成

- エンジニア向け
 - 機能やUIの仕様
 - 使用技術、内部構成、その採用理由
 - アーキテクチャMermaid図
 - 上手いかなかったアプローチと代替案
- 運用チーム向け
 - RAG技術、 Bedrock KnowledgeBase とは
 - ナレッジベース空間やその他インスタンスなどの作成方法をスクショ付きで全ステップ詳細に解説
 - 情報源マークダウンの作成方法、S3配置例
 - 月次コスト試算（平均質問回数、平均文字数などを仮定して算出）

課題、筆者の取り組み、結果など

課題	取り組み	結果や成果
RAG機構の学習・実装コストが高い	Bedrock KnowledgeBaseを採用	属人リスク低減、工数削減、他リポジトリへの再現性向上 / AWS料金微増
応答を詳細に制御・安定させるには	Bedrock Agentを追加導入し、より綿密な自然言語指示やテンプレ編集、その試行錯誤	応答要件の改善（導入前後で同じ質問などして検証。 / ただし完全な制御は未達成）

RAG関連コストが見えないと不安	日次・月次累計料金をSlackに通知＆S3に保存するCRON処理をLambdaで構築	継続運用性向上
機能的なチャットIFの迅速な構築と、各サイトデザインへの柔軟性を両立させるには	ヘッドレスUIライブラリ＋CSS。	機能系・状態管理系のFE工数大幅削減、他リポジトリへの再現性向上

1-2. タスク管理&通知社内システム開発【2025/06】

概要

全社的に「誰が、どういうタスクを持ち、今どういう状態にあるのか」の把握を容易にする、**Notionアプリ上でのタスク管理社内システム**。タスクのステータス変更およびコメント付与をトリガーとして、2秒以内に関連社員のSlackへ通知します。大まかなアーキテクチャは、任意のエンドポイントへPOSTリクエストを送信できる **Notion WebhookActions** という公式機能をクライアントとし、そのリクエストを受け取りさらに外部API各種を叩いて情報収集と通知を行う **Hono** 製の自前APIがバックエンドという構成です。筆者は技術選択とアーキテクチャ設計**以外**の仕様策定、API本体の実装、運用保守を担当しました。

使用技術

- フロントエンドFE：Notion , Notion WebhookActions , Slack , Slack BlockKit
- バックエンドBE：Hono , Notion API , Slack API
- インフラ/その他：Cloudflare Workers , Ngrok

チーム体制

役割	人数	担当業務	備考
ディレクター	2名	要件定義、調査など	他案件と兼任
デザイナー	1名	Figmaで通知UIデザイン作成	ディレクターが兼任
開発エンジニア（筆者）	2名	（技術選択、アーキテクチャ設計）、詳細仕様策定、実装、継続保守	アーキテクチャ設計までを担当した先輩開発者の休暇を機に、 筆者がその後引き継ぎ

筆者の担当業務詳細

- ディレクターと共に詳細仕様策定
 - タスク自体の項目（作業者、確認者、開始日/期限など）
 - タスクステータス種別（未着手、作業中、社内確認中など10種ほど）
 - それぞれのタスクステータス変更時、誰に通知を送るのか
 - どのような文言内容、SlackUIで通知するのか
- バックエンドAPI本体の実装
 - ステータス変更時用、コメント付与时用2つのAPIエンドポイント
 - タスクに関連する全社員を特定、情報取得、通知する各クラスやメソッドなど実装
 - Notion API , Slack API** と接続
 - デザイナー作成の通知UIを **Slack BlockKit** （実体はJSON）で構築
- 継続保守
 - Notion APIの破壊的変更リリースへの対応
 - 不具合時のデバッグ

課題、筆者の取り組み、結果など

課題	取り組み	結果や成果
実装時：――	仕様策定、API本体と通知UIの実装	1.5人日ほど想定より早く実装できた
保守時：――	ログ追跡で不具合特定・解消、利用APIの破壊的変更リリース対応など	一般的な保守を着実にこなせた

1-1. 建築関連会社仲介&管理Webアプリ受託開発【2024/08 - 2025/03】

概要

ご依頼主の建築資材商社様が保有する取引先ネットワークを活用した**業者および案件仲介Webアプリ**。発注業社（元請）と受注業者（下請）を、会社または案件単位でマッチングさせる**2つの仲介機能を主に提供する**ものです。ユーザー画面にて案件登録・検索・エントリーや、業者検索・マッチ申請などができます。それら情報を反映させた管理画面を元に、先方商社の営業ご担当者様が発注・受注業者双方に営業活動して仲介をクロージングする仕様のためCRMアプリでもあります。筆者は詳細仕様の検討と提案から、技術選択、両画面のFEとBE実装、ドキュメント作成、継続保守まで一貫して開発業務全般を主担当しました。

使用技術

- フロントエンドFE：React18 , Next.js14 , SCSS , TailwindCSS , Shadcn/ui , TanStack Query , TanStack Table , Yup , ReactHookForm
- バックエンドBE：Next.js14(ServerActions) , Supabase ORM/Auth , AmazonSES , GoogleAppScript
- インフラ/その他：Cloudflare Workers（アプリ本体） , OpenNext , Supabase DB (PostgreSQL) , MicroCMS , Figma

チーム体制

役割	人数	担当業務	備考
ディレクター	1名	依頼企業様との要求整理、全体的な要件定義、管理画面ワイヤーフレームWFの作成、開発者との仕様協議など	担当者の本業は動画制作とマークアップであり、 アプリ開発およびディレクションは未経験での挑戦 。加えて 他の受託開発案件のディレクションやHP案件のマークアップ業務とも同時に兼任 。本案件中盤でご退職
マークアップエンジニア	2名	ユーザー画面の静的ページマークアップ、筆者のHTMLセマンティクスとCSSの改善、アニメーション追加、CMS経由のブログ記事ページ、サイトマップやメタタグなどSEO関連実装	他案件と兼任
開発エンジニア（筆者）	1名	上記マークアップエンジニアの担当範囲と、RDB設計叩き台 以外 の開発関連業務全て。下記セクションにて詳述	本案件を専任。開発職の先輩（本案件終盤でご退職）にも適宜相談
マーケター / CS	2名	ページ内文言・メール文言などの検討、マニュアル資料の作成、問い合わせ一次対応など	他案件と兼任。本案件終盤で1名ご退職
デザイナー	2〜3名	URLパスの策定、ワイヤーフレーム作成、UIデザイン作成	他案件と兼任。本案件終盤で1名ご退職

開発環境の前提

- 会社としてWebアプリ開発の実績と機会自体が少ないため、初実務の筆者含めて社員の知識や経験値、人員、工期など根本的なリソースが慢性的に不足。そのため実装レベルまで仕様が事前に検討・ドキュメント化されることはほぼ無いので、**開発者はディレクターにその都度ヒアリングし、詳細な仕様とアプローチを自ら検討＆提案して進めるのが基本**でした（詳細は下記セクションにて）。
- 筆者含めてテストコード作成およびコードレビューを行える人材がいなかったため、各自ローカルで動作確認後セルフマージする運用（会社にGitが導入されて当時1年未満）。リリース前に全ページで手動E2Eテスト？を実施するのが通例のようでした。

- RDBテーブル設計書の叩き台作成と、メイン技術に **TypeScript** , **Next.js** , **Supabase** を採用するという意思決定の2点が、**筆者が参加する以前に**既になされていました（会社として初めての **Next.js** アプリ受託開発だが、担当者が退職したため筆者が急遽採用されたという経緯）。
- **別の受託開発案件が同時期に開始した**影響もあり、最終的には静的ページ以外を筆者が実装しました。

筆者の担当業務詳細

- ディレクターと共に要件と実装イメージのすり合わせ
 - 大まかな仕様を口頭やチャットでヒアリング、整理して適宜相談
 - 機能やUI挙動の詳細仕様を検討・提案（下記①）
- RDBテーブル設計の調整と保守、Supabase立ち上げと管理など
 - リレーション未遂の修正、カラムやテーブルの命名・データ型改善
 - **Supabase ORM/DB/Auth** の学習
 - 開発用と本番用のプロジェクト立ち上げや管理雑務
- ユーザー画面、管理画面のFEとBE実装（下記②③）
 - 要件達成やUX向上のための技術選択と学習
 - ディレクトリ設計（関連物を最大限コロケーション）
 - Reactコンポーネント設計（クラサバの境界線意識、RSC維持などの基本）
 - Figmaを元にUIマークアップ
 - FE状態管理、フォームバリデーション
 - 認証と認可（ユーザー権限に応じた可変UIや閲覧規制など）
 - 各種CRUD操作
 - メール送信、GASによる集計など
- ホスト先PaaSのセットアップ、および2度の移行
 1. 月額シートコスト削減打診を受け、 **Vercel** から **Cloudflare Pages** へ移行。Nodeからエッジへのランタイム移行に伴うコード修正、Pagesデプロイ用ライブラリ導入など実施
 2. **Next.js** の脆弱性発表に伴い、新バージョン不対応のPagesデプロイ用ライブラリ入れ替えと **Cloudflare Workers** への移行。 **OpenNext** ライブラリの調査・導入など実施
- 属人性低下させるためドキュメント作成（下記④）
- リリース時にプロダクト紹介ご訪問
- リリース後の追加要望対応など継続保守

①詳細仕様の検討と提案

- 主要機能例
 - 各一覧ページ検索機能の絞り込み仕様（各項目と、その項目内の選択をクエリにどう反映するか）
 - 各一覧ページ表示仕様（既定の絞り込みや並び順は何かいいか）
 - 外部APIによる住所の座標変換時の仕様（「正確な座標」要件を画一的に保証できる全国住所の詳細度とは）
 - 会社・案件マッチングステータス各種の分岐表示仕様（ネストif文のようなもの）など
- UIワイヤーフレーム改善例
 - 管理画面の各種検索一覧ページにて、絞り込み項目セレクトボックスの複数追加（当初は文字列検索インプットのみ）、各項目に昇降順切り替え追加、など
 - 管理画面のナビゲーションUIにおいて、ほぼデッドスペースになるサイドバーではなく、画面上部に水平タブを配置
 - ユーザー画面の検索フォームにて、ログイン中ユーザー情報の自動記入ボタン追加など

②管理画面の実装（FEとBE）

全10画面ほど

- 業者（ユーザー）一覧、検索
- 業者詳細、編集
- 業者マッチング申請一覧、検索
- 各詳細、編集
- 案件一覧、検索
- 各詳細（エントリー状況など）、編集
- ログイン、削除済みユーザー一覧・詳細などその他ページ

③ユーザー画面の実装（FEとBE）

全10画面ほど

- ユーザー（業者）登録画面2件
- 案件登録
- 案件一覧、検索
- 案件詳細、エントリー
- 業者検索
- 業者検索結果、近い順に並び替え、マッチ申請
- マイページ
 - 案件登録履歴
 - 案件エントリー履歴
 - 業者マッチング履歴
 - ユーザー情報閲覧、編集
- 問い合わせ、メール送信・GAS集計
- その他

④ドキュメント作成

- エンジニア向け
 - FE, BEそれぞれの使用技術
 - ディレクトリ設計
 - Next.jsでのBE実装方法（FEからのServerAction使用法など）
 - Supabaseの基礎知識など
 - 認証関連データを管理するAuthスキーマ
 - Supabaseインスタンスは絶対にサーバーサイドで初期化する
 - 無料プランの注意点など
 - **Cloudflare Workers** , **OpenNext** 関連
- 全社員向け
 - アプリ概要
 - 開発体制、背景など
 - ユーザー権限仕様

- 全機能ページの紹介（表示仕様・検索仕様など内部ロジックも）
- 社内ドキュメントリンク集
- 定期保守作業の手順書など

課題、筆者の取り組み、結果など

課題	取り組み	結果や成果
動的機能UIを短納期で開発するには	独自デザイン要件が強いユーザー画面にはヘッドレスUIライブラリを、一方管理画面などにはスタイリング済みUIライブラリの使用を提案。どちらも実際に使用して実装例を残す	アプリUI開発の工期大幅短縮、ビジネスメリットの周知 / ただし短納期は不変なので、実装者の業務外学習に依存（この例に限らず）
コード随所で同様なTS型が徐々に増加。またTS型エラーを放置・無理に突破する風潮が常態化しそうだった	なるべくTS型は自作せずにDBスキーマから自動生成、特定場所に保存・更新しながら使い回す運用に変更。参加エンジニアへ周知と徹底	デバッグ時間削減、IDE補完もパチパチに効いて開発生産性向上、知見不足かつ短納期は開発者モラルを大きく低下させる？という気づき
仕様が不明、未定義	自ら要求ヒアリング、仕様検討・調査・提案、実装まで完遂	単独でも責任を持って自走し、プロジェクトを進捗 / 筆者の属人性増加
属人リスクを下げるには	全社向け・エンジニア向けに丁寧にドキュメント作成	属人リスク低下、引継ぎ保守性など向上
筆者のタスク粒度が大きく、進捗や作業量を周囲が把握しにくい	反省点 ：お互いの慢性的な時間不足からディレクターとは口頭会議も多く、テキストでも文章量が多いため（勝手に周囲に配慮したつもりで）頻繁に個別チャットしていた	学び ：開発者の作業範囲や実際の工数が理解されにくい現場では、むしろ議論や提案は常にオープンにすべきだった。

1-ex. その他並行業務

既存開発案件の引継ぎ保守【2025/10 - 2026/02】

リリースから既に1年弱経過したトランクルーム予約ポータルサイト案件を引継ぎ、**バグ発見と修正、社内・先方確認フローの改善、定型作業**などを行いました。上記の1案件目と同時期に開始したFE受託開発案件です（BEは外注）。

- バグ発見・修正
 - FEとAPI間で部屋ID仕様の考慮漏れが原因で、約10店舗150部屋以上がリリース以来選択不可状態だったことを発見。該当コードを修正し全件解消
- 社内・先方確認フローの改善
 - 従来は作業ブランチをステージング環境にマージしてから社内と先方へ確認出し、修正があれば再度作業ブランチ作成を行っていた。そこでコードプッシュごとに作業ブランチ単位でビルド＆プレビュー環境URLが発行されるようホスト先PaaS(**AWS Amplify**)を設定。これにより先方確認まで完了後にマージできるようになりブランチ運用を効率化

Web制作案件ごとにPaaSとDNSの管理【2025/06 - 2026/01】

社内の全Web制作案件ごとにホスト先PaaS（ **Cloudflare Pages ）とDNSの管理**を担当しました。ドメイン名管理やメールサーバーは別の大手レンタルサーバーのため、やや複雑になりがちな要求を慎重にヒアリング・各種DNSレコード設定をしました。

1. Cloudflareインスタンス作成、Github接続
2. 社長からドメイン関連要件をヒアリング
3. 各種DNSレコード設定
4. Pages側リダイレクトルール設定
5. 自動ビルドするプレビュー環境URLの認証設定など
6. サイト公開時の切り替え作業

業務外活動

ハンズオン学習

主にハンズオン学習目的で作成したリポジトリたちを一部Publicにして下記に記載。本来公開するほどのレベルではないので、あくまで個人的に取り組んできた技術分野の例として紹介します(作成日降順)。

リポジトリ名	使用技術	概要と備考
resume (準備中)	TS,React19,TanStack Router, TailwindCSS,Vite,Github Actions,Github Pages,etc.	マークダウンで作成した職務経歴書をWebサイト化して、スタイリングやURL共有を可能にするためのプロジェクト 。自動デプロイなど継続更新するハードルを下げる仕組みは積極的に導入していく予定です
full-tanstack-starter	TS,React19,TanStack Start/Query/Form,TailwindCSS,BetterAuth,DrizzleORM,Vite, NeonDB,Cloudflare,Github Actions etc.	フルスタックWebアプリ開発に必要な諸々を素早く展開するために、個人的に使いたい各分野の主要ライブラリを事前に組み合わせたテンプレ。 今後のAIコーディングツール使用を前提に、ドキュメント運用手法を学習中 です。使用技術や設計は随時変更予定です
discord-clone (停止中)	TS,React18,Next.js13, TailwindCSS,Zustand,Clerk, PrismaORM,Socket.io,Docker, Render,etc.	エンジニア転職用のポートフォリオとして作成した最初のWebアプリ。機能は基本的なチャットIF、コミュニティや会話チャンネルの作成・招待追放・管理などのCRUD操作、WebSocketによるリアルタイム通信、ビデオ通話、フルレスポンスUI、ダークモード、ランディングページなどです。未経験とはいえ チーム開発や保守性への意識も妥協せず、有効そうな仕組みを適宜リサーチ・導入するなど当時の限界まで様々な技術的挑戦をしました 。具体的には、アプリ本体と本番DB両方をDockerコンテナ化して可搬性向上、開発環境も丸ごとDockerコンテナ化して再現性を高めるDevContainerの作成、Github Projectsを使用してタスクごとにバックログ作成＆管理してみた一人擬似アジャイル？などです。また最初から全ての記録を英語にしていたことが功を奏し、海外の同じ駆け出しエンジニア数名からとても感謝されたことも非常に嬉しい出来事でした
next13-nestjs-graphql	TS,React18,Next.js,NestJS, PrismaORM,GraphQL,Apollo, Turborepo,Docker,etc.	Next.js(FE)やNestJS(BE)など主要フレームワークの学習に加えて、当時頻繁に目にしていた「GraphQL」や「モノレポ構成」なるものに興味を持ってしまい、結果半年以上費やしてしまった成れの果て。当然ですが GraphQLは、単一言語でのフルスタックWebアプリ開発には過剰な技術 。薄々理解はしていたためしばらく経ったのち反省。「熱意だけはある未経験者が独学で技術を学ぶ際のリスク」を経験できたことを教訓にしたいです

ブログ投稿

個人名では特になし（1社目では5本投稿）

OSS

特になし

イベント登壇

特になし